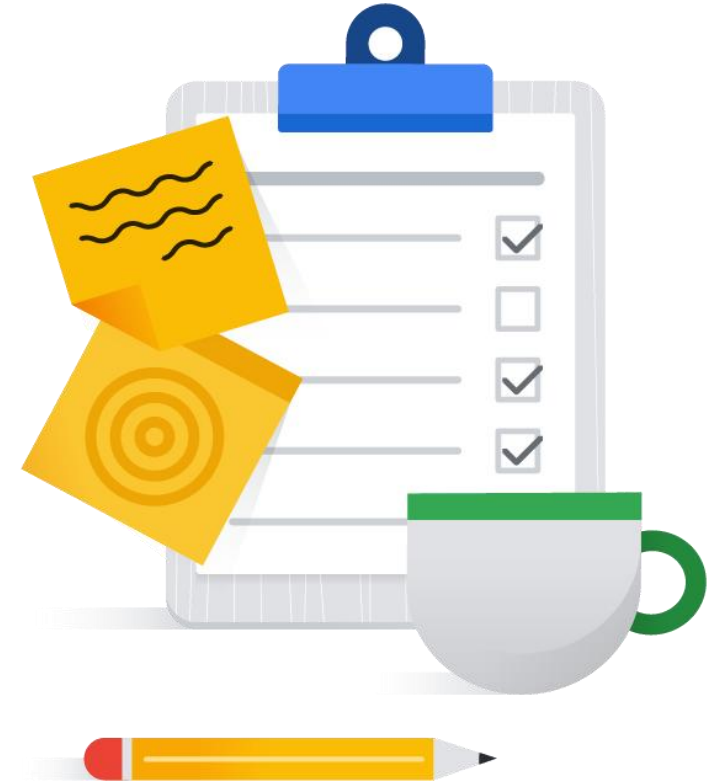


BigQuery ML

11

Topics

- | | |
|----|--|
| 01 | Introduction to BigQuery ML |
| 02 | How to Make Predictions with BigQuery ML |
| 03 | How to Build and Deploy a Recommendation System |
| 04 | How to Build and Deploy a Demand Forecasting Solution with BigQuery ML |



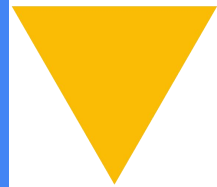
Topics

05 Time-Series Models with BigQuery ML

06 BigQuery ML Explainability



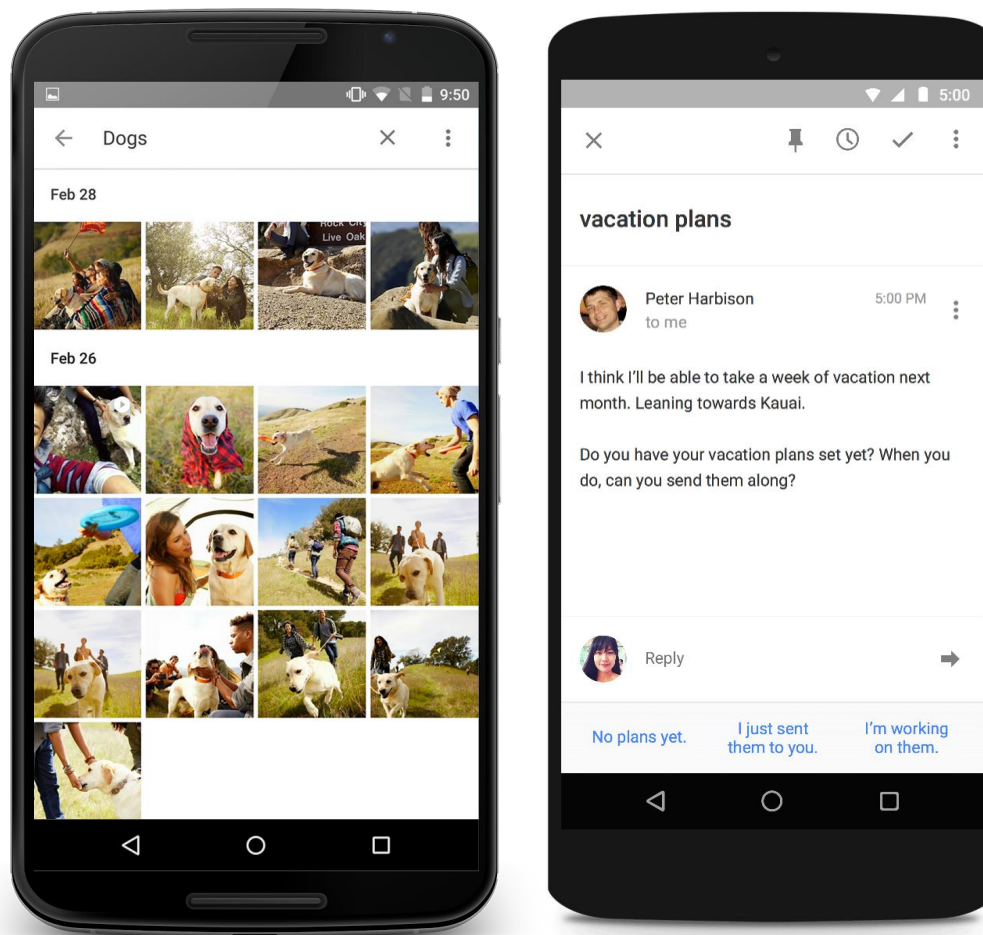
01



Introduction to BigQuery ML

When you hear “AI or ML,” you probably think of:

Image models
Sequence models
Neural networks



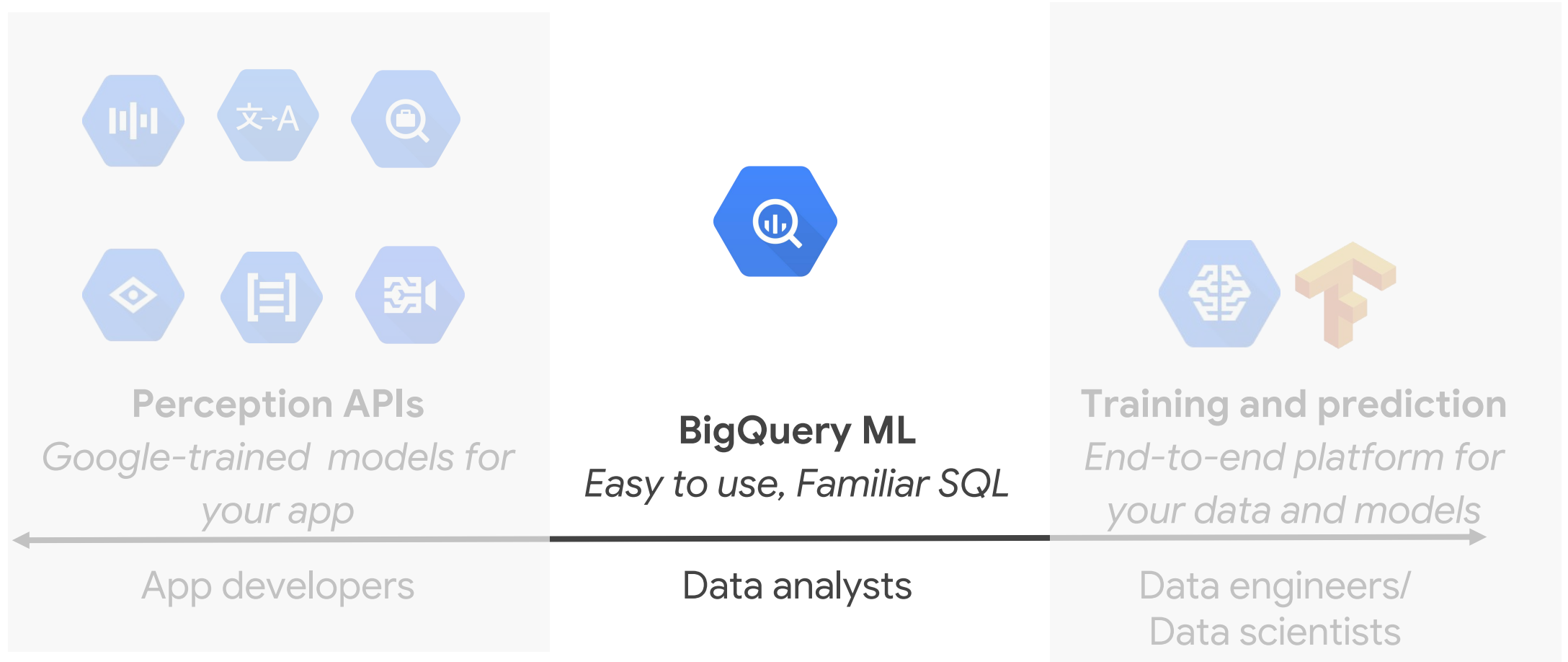
The most common ML models at Google are those that operate on structured data

ML on structured data drives value

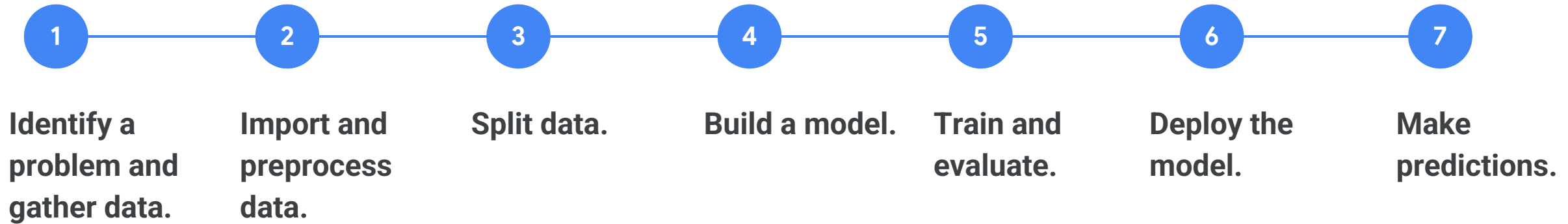
Source (2017):
<https://cloud.google.com/blog/big-data/2017/05/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu>

Type of network	# of network layers	# of weights	% of deployed models
MLP0	5	20M	61%
MLP1	4	5M	
LSTM0	58	52M	29%
LSTM1	56	34M	
CNN0	16	8M	5%
CNN1	89	100M	

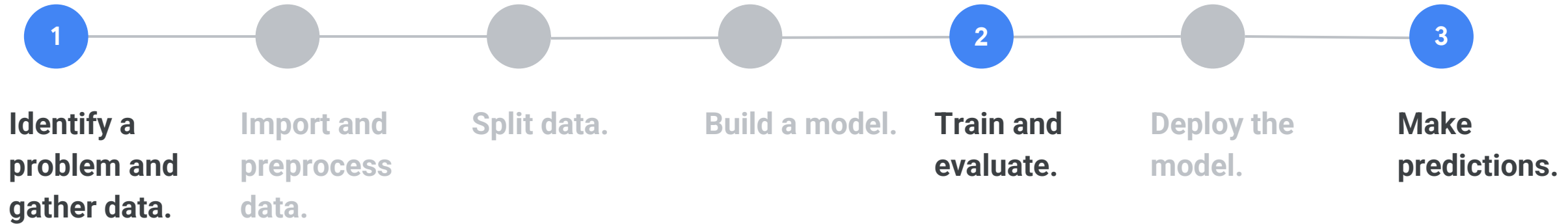
BigQuery ML is a way to easily build machine learning models



Custom model **without** BigQuery ML



Custom model **with** BigQuery ML



BigQuery ML



Execute ML models without moving data from BigQuery.



Iterate on models in SQL in BigQuery to increase development speed.



Automate common ML tasks and hyperparameter tuning.

Supported BigQuery ML models

Classification

- Logistic regression
- DNN classifier (TensorFlow)
- XGBoost
- AutoML Tables

Other models

- k-means clustering
- Time series forecasting
- Recommendation: Matrix factorization

Regression

- Linear regression
- DNN regressor (TensorFlow)
- XGBoost
- AutoML Tables

Model import/export

- TensorFlow models for batch and online prediction

BigQuery model inference

- ML.PREDICT
- ML.FORECAST
- ML.RECOMMEND
- ML.DETECT_ANOMALIES
- ML.RECONSTRUCTION_LOSS



Key phases of a BigQuery ML project

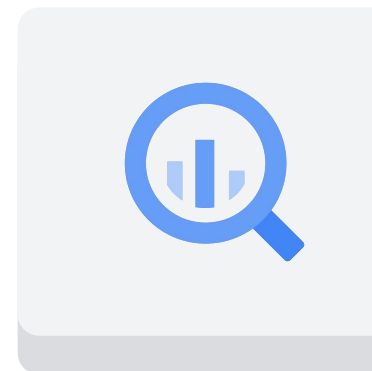
01 Extract, transform, and load data into BigQuery.

02 Select and preprocess features.

03 Create the model inside BigQuery.

04 Evaluate the performance of the trained model.

05 Use the model to make predictions.



BigQuery ML

How to make predictions with BigQuery ML



How to Make Predictions with BigQuery ML



Table info

Table ID	nyc-tlc.yellow.trips
Table size	129.72 GB
Long-term storage size	129.72 GB
Number of rows	1,108,779,463

pickup_datetime	dropoff_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	rate_code	passenger_count
2010-03-04 00:35:16 UTC	2010-03-04 00:35:47 UTC	-74.035201	40.721548	-74.035201	40.721548	1	1
2010-03-15 17:18:34 UTC	2010-03-15 17:18:35 UTC	0.0	0.0	0.0	0.0	1	1
2015-03-18 01:07:02 UTC	2015-03-18 01:07:07 UTC	0.0	0.0	0.0	0.0	1	5
2015-03-09 18:24:03 UTC	2015-03-09 18:25:37 UTC	-73.93724822998047	40.758201599121094	-73.93726348876953	40.7581901550293	1	1
2010-03-06 06:33:41 UTC	2010-03-06 06:36:06 UTC	-73.785514	40.6454	-73.784564	40.648681	1	2
2013-08-07 00:42:45 UTC	2013-08-07 00:58:43 UTC	-74.025817	40.763044	-74.046752	40.78324	5	1
2015-04-26 02:56:37 UTC	2015-04-26 03:00:01 UTC	-73.98765563964844	40.77165603637695	-73.98755645751953	40.771751403808594	1	1
2015-04-29 18:45:03 UTC	2015-04-29 18:49:01 UTC	0.0	0.0	0.0	0.0	1	1
2010-03-11 21:24:48 UTC	2010-03-11 21:46:51 UTC	-74.571511	40.9108	-74.628928	40.964321	1	1
2013-08-24 01:58:23 UTC	2013-08-24 01:58:23 UTC	-73.972171	40.759439	0.0	0.0	5	4

Select data



[Photo from Unsplash](#)

```
SELECT
    fare_amount,
    pickup_longitude,
    pickup_latitude,
    dropoff_longitude,
    dropoff_latitude,
    passenger_count

FROM
    `nyc-tlc.yellow.trips`
```

Build and train with **CREATE MODEL**



[Photo from Unsplash](#)

```
CREATE OR REPLACE MODEL
  mydataset.model_linreg

OPTIONS(
  input_label_cols=[ 'fare_amount' ],
  model_type='linear_reg') AS

SELECT
  fare_amount,
  pickup_longitude,
  pickup_latitude,
  dropoff_longitude,
  dropoff_latitude,
  passenger_count

FROM
  `nyc-tlc.yellow.trips`
```

Evaluate with **ML.EVALUATE**



[Photo from Unsplash](#)

```
SELECT  
  *
```

```
FROM  
  ML.EVALUATE(  
    MODEL mydataset.model_linreg  
  )
```

Use the model with **ML.PREDICT**



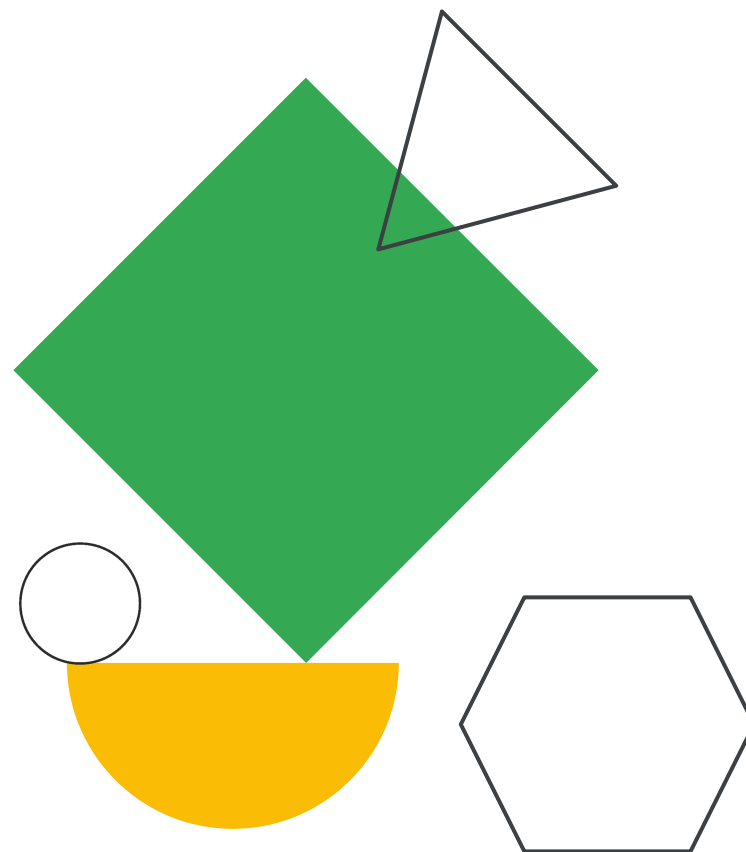
[Photo from Unsplash](#)

```
SELECT
    *

FROM
    ML.PREDICT(MODEL mydataset.model_linreg,
    (
        SELECT
            fare_amount,
            pickup_longitude,
            pickup_latitude,
            dropoff_longitude,
            dropoff_latitude,
            passenger_count
        FROM
            `nyc-tlc.yellow.trips`
    ))
```

Demo

Linear regression model: Taxi
fare





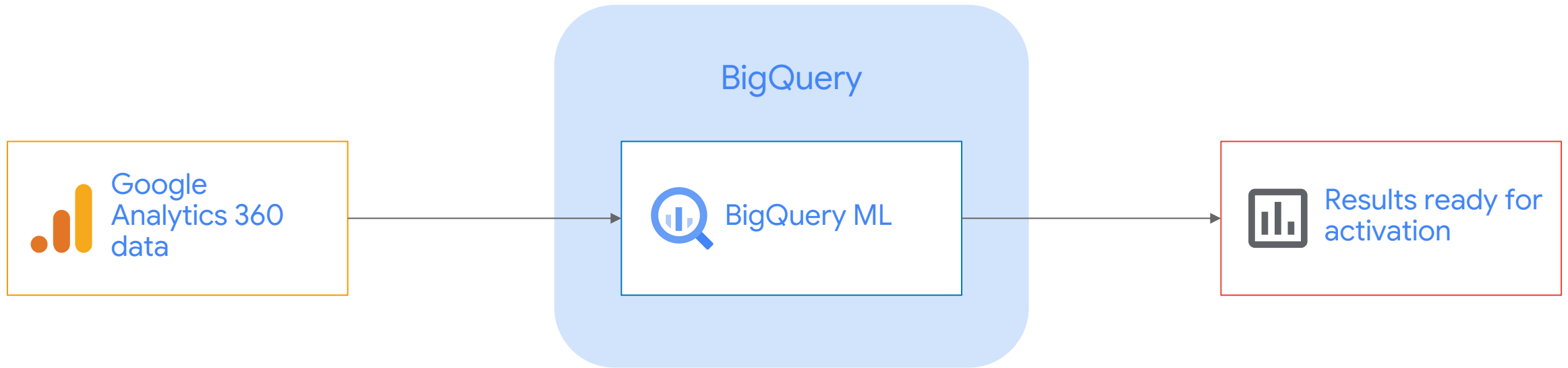
03



How to Build and Deploy a Recommendation System



Businesses need to build **recommendation systems** to improve **conversions** and **clickthrough-rates** and build **customer loyalty**.



- **Prepare** your training data in BigQuery.
- **Train** a recommendation system with BigQuery ML.
- Use the predicted recommendations **in production**.

Preparing the training data

How do you know whether a **user** likes a **product** ?

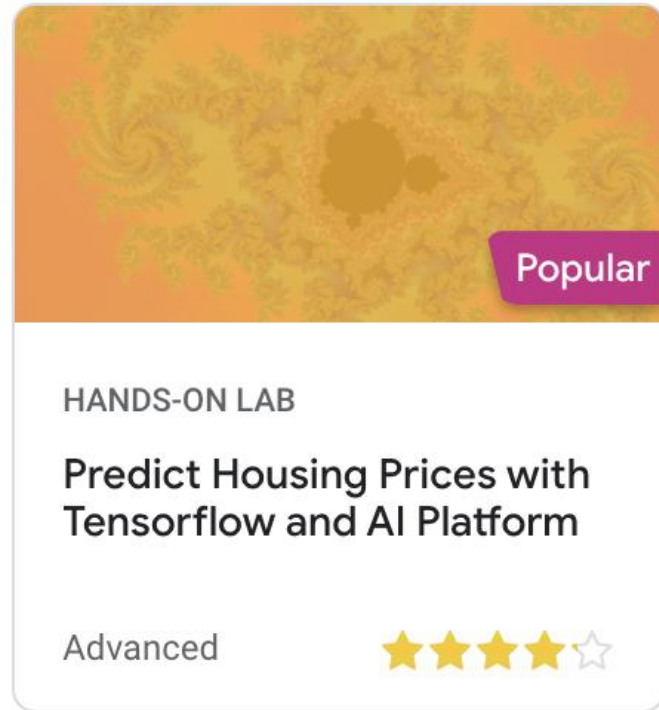
item

service

news article

movie

Explicit feedback

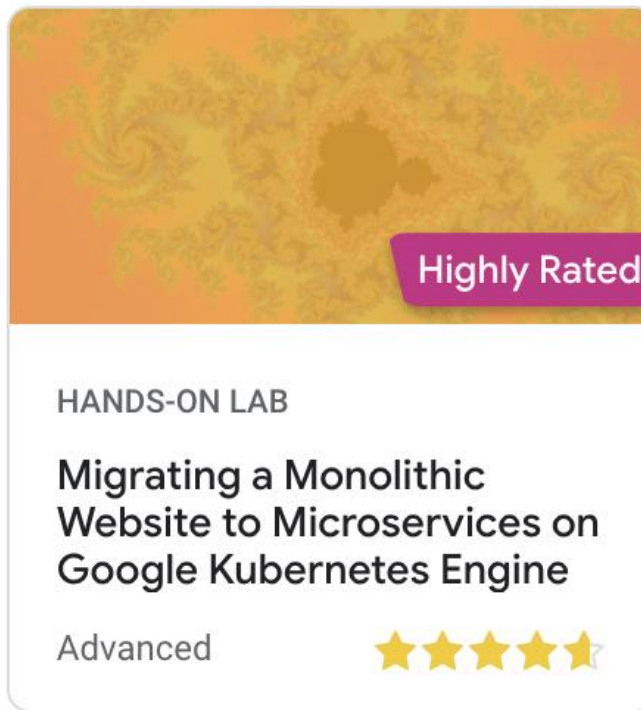


Popular

HANDS-ON LAB

Predict Housing Prices with Tensorflow and AI Platform

Advanced 

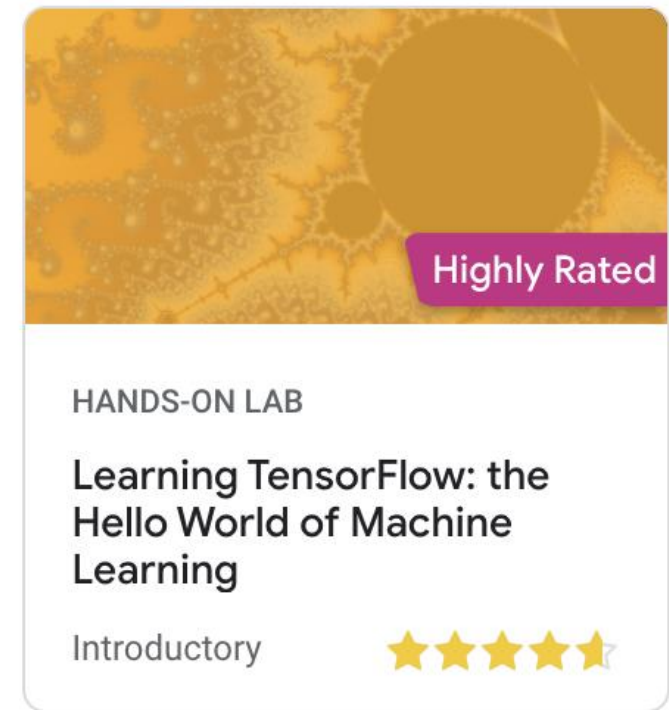


Highly Rated

HANDS-ON LAB

Migrating a Monolithic Website to Microservices on Google Kubernetes Engine

Advanced 



Highly Rated

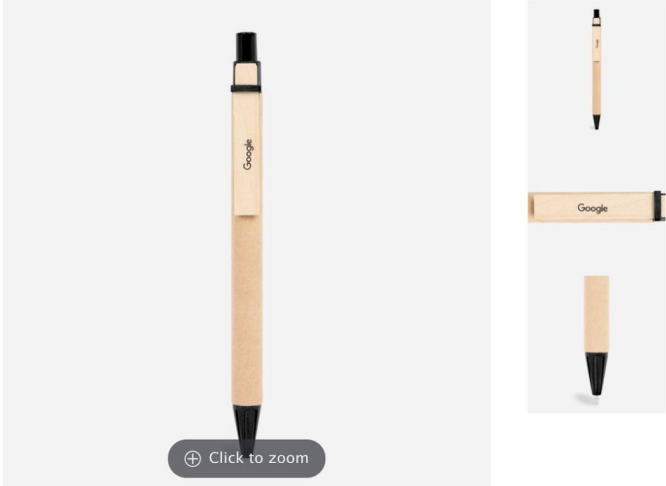
HANDS-ON LAB

Learning TensorFlow: the Hello World of Machine Learning

Introductory 

Implicit feedback

For example, **amount of time spent viewing a product**



Google Recycled Pen Black

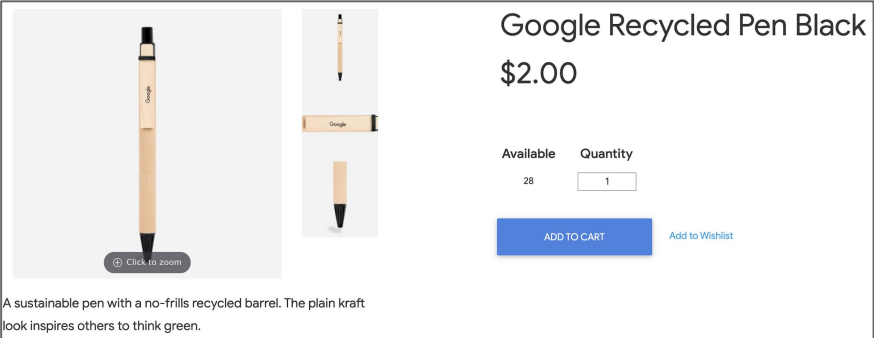
\$2.00

Available 28 Quantity 1

ADD TO CART Add to Wishlist

A sustainable pen with a no-frills recycled barrel. The plain kraft look inspires others to think green.

Sample Google Analytics 360 data
from the Google Merchandise Store



Available on
BigQuery Public Datasets

Pre-processed training data

	visitorId	itemId	session_duration
0	5468761641774363851-1	GGOEGAAX0031	59
1	7186762836575002506-1	GGOEGAAX0031	5787
2	1827763305655925232-3	GGOEGAAX0031	6423
3	3863873694540855771-4	GGOEGAAX0031	6168
4	2046797444872582027-2	GGOEGAAX0031	6891
5	5885261202339404877-2	GGOEGAAX0031	85256
6	0097216552247524030-5	GGOEGAAX0031	39980
7	860497822069663220-1	GGOEGAAX0031	2486
8	8960897138106393989-1	GGOEGAAX0031	11641
9	9681217015581152227-1	GGOEGAAX0031	6398

<https://console.cloud.google.com/marketplace/details/obfuscated-ga360-data/obfuscated-ga360-data?filter=solution-type:dataset>

Training a recommendation system using **matrix factorization** with BigQuery ML

Matrix factorization

Matrix factorization is a class of collaborative filtering algorithms used in recommender systems.

Matrix factorization algorithms work by decomposing the user-item interaction matrix into the product of two lower dimensionality rectangular matrices.

	 Harry Potter	 The Triplets of Belleville	 Shrek	 The Dark Knight Rises	 Memento
	✓		✓	✓	
		✓			✓
	✓	✓	✓		
				✓	✓

	visitorId	itemId	session_duration
0	5468761641774363851-1	GGOEGAAX0031	59
1	7186762836575002506-1	GGOEGAAX0031	5787
2	1827763305655925232-3	GGOEGAAX0031	6423
3	3863873694540855771-4	GGOEGAAX0031	6168
4	2046797444872582027-2	GGOEGAAX0031	6891
5	5885261202339404877-2	GGOEGAAX0031	85256
6	0097216552247524030-5	GGOEGAAX0031	39980
7	860497822069663220-1	GGOEGAAX0031	2486
8	8960897138106393989-1	GGOEGAAX0031	11641
9	9681217015581152227-1	GGOEGAAX0031	6398

Training data

```
SELECT
  *
FROM
  bqml.aggregate_web_stats
```


Build and train with CREATE MODEL

	visitorId	itemId	session_duration
0	5468761641774363851-1	GGOEGAAX0031	59
1	7186762836575002506-1	GGOEGAAX0031	5787
2	1827763305655925232-3	GGOEGAAX0031	6423
3	3863873694540855771-4	GGOEGAAX0031	6168
4	2046797444872582027-2	GGOEGAAX0031	6891
5	5885261202339404877-2	GGOEGAAX0031	85256
6	0097216552247524030-5	GGOEGAAX0031	39980
7	860497822069663220-1	GGOEGAAX0031	2486
8	8960897138106393989-1	GGOEGAAX0031	11641
9	9681217015581152227-1	GGOEGAAX0031	6398

Training data

```
CREATE OR REPLACE MODEL
  bqml.retail_recommender

OPTIONS(
  model_type='matrix_factorization',
  user_col='visitorId',
  item_col='itemId',
  rating_col='session_duration',
  feedback_type='implicit'
) AS

SELECT
  *
FROM
  bqml.aggregate_web_stats
```

Requires flat-rate pricing: <https://cloud.google.com/bigquery/docs/reservations-intro>

Evaluate the model with **ML.EVALUATE**

```
SELECT
  *
FROM
  ML.EVALUATE(
    MODEL bqml.retail_recommender
  )
```

Row	mean_average_precision	mean_squared_error	normalized_discounted_cumulative_gain	average_rank
1	0.011463546060150412	1.0999974250389957E8	18.68240759482596	0.28084605495250364

Predict for a
single user with
ML.RECOMMEND

```
DECLARE MY_VISITORID STRING DEFAULT "6499749315992064304-2";

SELECT
  *
FROM
  ML.RECOMMEND(
    MODEL bqml.retail_recommender,
    (SELECT MY_VISITORID as visitorID)
  )
ORDER BY predicted_session_duration_confidence DESC

LIMIT 5
```

Row	predicted_session_duration_confidence	visitorId	itemId
1	35344.75190048335	0824461277962362623-1	GGOEGAAX0074
2	28775.356706353305	0824461277962362623-1	GGOEGBJC076099
3	26283.729992603585	0824461277962362623-1	GGOEAFKQ020499
4	24583.28072228624	0824461277962362623-1	GGOEGAAX0289
5	23462.16560910448	0824461277962362623-1	GGOEGOAB021699

Batch predictions for all users with **ML.RECOMMEND**

Row	visitorId	itemId	predicted_session_duration_confidence
1	0000010278554503158-1	GGOEGBJC076099	33266.005663956515
2	0000010278554503158-1	GGOEADHB014799	23798.823558455344
3	0000010278554503158-1	GGOEYHPB072210	23317.03648749228
4	0000010278554503158-1	GGOEGEVA022399	23045.588017967602
5	0000010278554503158-1	GGOEGAAX0074	22231.48418727726
6	0000020424342248747-1	GGOEGBJC076099	31222.7514436289
7	0000020424342248747-1	GGOEYHPB072210	29845.991045227882
8	0000020424342248747-1	GGOEGODR017799	28248.607012765326
9	0000020424342248747-1	GGOEGEVA022399	28086.011190871777
10	0000020424342248747-1	GGOEGOBG023599	26348.173682193796

**How to use the
predicted recommendations
in **production** ?**

1. Export recommendations for ad retargeting campaigns.

clientId	LikelyToBuyProductA
123	0.70
345	0.90

*For product campaigns,
create a new column for “likelihood to purchase”
based on the predicted recommendations,
and then import into Google Analytics to create new campaigns.*

2. Connect with your CRM for personalized emails.

“Hi Maria, you might be interested in {A}, {B}, {C}.”



How to Build and Deploy a Demand Forecasting Solution with BigQuery ML

Transportation

→ Predict ticket sales



<https://unsplash.com/photos/9qQTUYm4ss4>

Media/Gaming

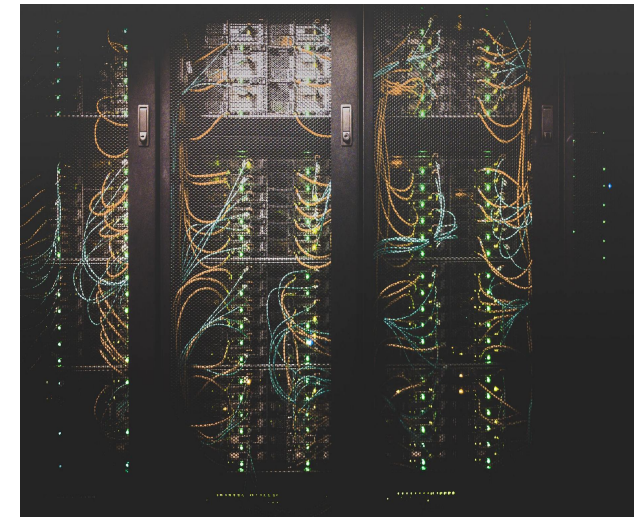
→ Predict # active players/time
→ Predict content viewership



<https://unsplash.com/photos/4P0zdOSstql>

Telecommunications

→ Predict network traffic

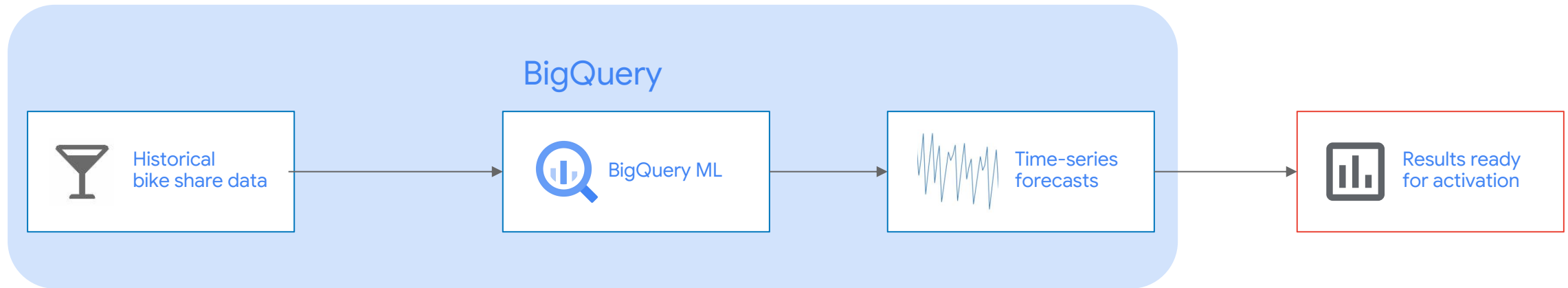


<https://unsplash.com/photos/M5tzZtFCOfs>

Retail



<https://unsplash.com/photos/nHRXNv2qeDE>



1. **Prepare** the training data in BigQuery.
2. **Train and evaluate** a time-series model with BigQuery ML.
3. **Visualize** the forecasts in a dashboard.
4. **Schedule and automate** model retraining.

Preparing the **training data**

Bike share data

Transactional data

Row	trip_id	subscriber_type	bikeid	start_time	start_station_id	start_station_name	end_station_id	end_station_name	duration_minutes
1	9900289692	Walk Up	248	2015-10-02 21:12:01 UTC	1006	Zilker Park West	1008	Nueces @ 3rd	39
2	9900285987	24-Hour Kiosk (Austin B-cycle)	446	2014-10-26 15:12:00 UTC	2712	Toomey Rd @ South Lamar	2712	Toomey Rd @ South Lamar	31
3	9900285989	24-Hour Kiosk (Austin B-cycle)	203	2014-10-26 15:12:00 UTC	2712	Toomey Rd @ South Lamar	2712	Toomey Rd @ South Lamar	31
4	9900285991	24-Hour Kiosk (Austin B-cycle)	101	2014-10-26 15:12:00 UTC	2712	Toomey Rd @ South Lamar	2712	Toomey Rd @ South Lamar	30
5	9900286140	24-Hour Kiosk (Austin B-cycle)	242	2014-10-26 18:12:00 UTC	2541	State Capitol @ 14th & Colorado	2541	State Capitol @ 14th & Colorado	19

```
`bigquery-public-data.austin_bikeshare.bikeshare_trips`
```

Training data

Row	no_bikes_difference	station_name	day
1	77	Zilker Park West	2015-10-02
2	24	Zilker Park West	2015-10-03
3	-17	Lavaca & 6th	2015-10-03
4	-9	Waller & 6th St.	2015-10-03
5	9	11th & San Jacinto	2015-10-03
6	-11	Waller & 6th St.	2015-10-09
7	24	Zilker Park West	2015-10-09
8	-45	Zilker Park West	2015-10-10
9	10	Nueces @ 3rd	2015-10-10
10	-30	Zilker Park West	2015-10-11

```
SELECT
```

```
no_bikes_taken - no_bikes_returned AS no_bikes_difference,  
start_station_name AS station_name, start_bikes.day AS day
```

```
FROM (
```

```
  SELECT
```

```
    COUNT(*) AS no_bikes_taken, start_station_name,  
    EXTRACT(DATE FROM start_time) AS day
```

```
  FROM
```

```
    `bigquery-public-data.austin_bikeshare.bikeshare_trips`
```

```
  GROUP BY
```

```
    start_station_name, day ) AS start_bikes
```

```
JOIN (
```

```
  SELECT
```

```
    COUNT(*) AS no_bikes_returned, end_station_name,  
    EXTRACT(DATE FROM start_time) AS day
```

```
  FROM
```

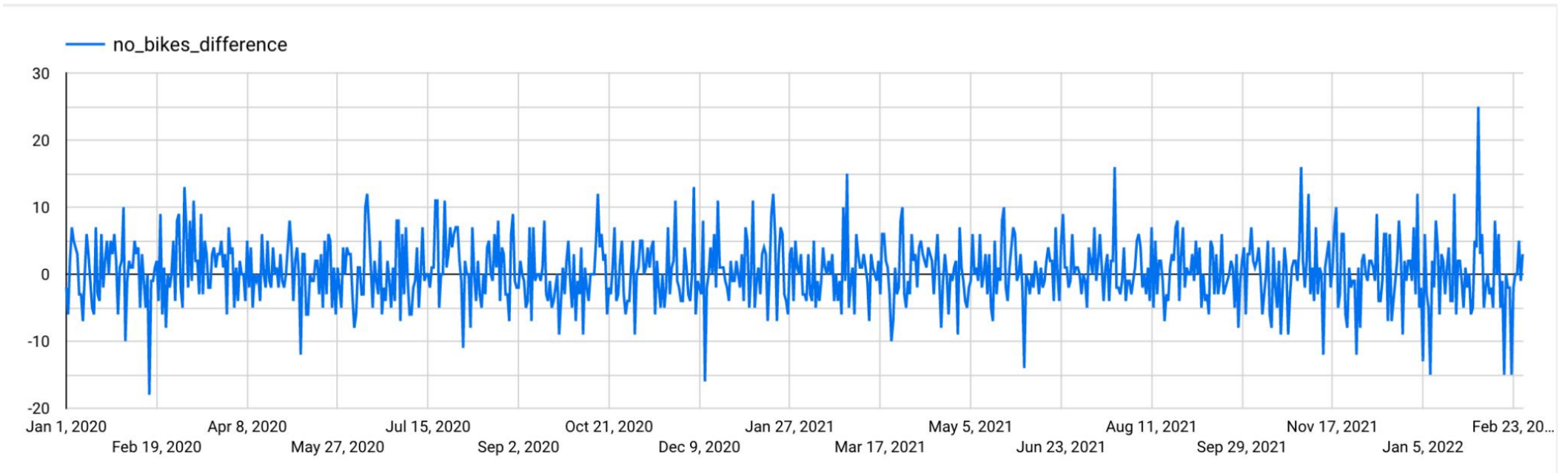
```
    `bigquery-public-data.austin_bikeshare.bikeshare_trips`
```

```
  GROUP BY
```

```
    End_station_name, day) AS end_bikes
```

```
ON
```

```
(end_station_name=start_station_name AND end_bikes.day =  
start_bikes.day);
```

Filter: date greater than or equal to 01/01/2020.

Creating the time-series model in SQL with BigQuery ML

A blue square containing the white text '05'. To the right of the square is a yellow downward-pointing triangle.

05

Time-Series Models with BigQuery ML

Build and train with CREATE MODEL

Row	day	station_name	no_bikes_difference
1	2014-10-04	Mobile Station @ Bike Fest	96
2	2015-10-02	Zilker Park West	77
3	2015-10-03	Pfluger Bridge @ W 2nd Street	50
4	2015-03-15	Mobile Station	45
5	2018-03-21	Nueces & 26th	44
6	2017-10-07	City Hall / Lavaca & 2nd	44
7	2015-10-10	Pfluger Bridge @ W 2nd Street	44
8	2018-03-01	22nd & Pearl	43
9	2018-04-09	22nd & Pearl	41
10	2018-04-19	22nd & Pearl	41
11	2017-10-14	Pfluger Bridge @ W 2nd Street	41
12	2016-10-02	Pfluger Bridge @ W 2nd Street	41

training_data

```
SELECT
    day,
    station_name,
    no_bikes_difference
FROM
    bikeshare_bqml.training_data;
```

Build and train with **CREATE MODEL**

Row	day	station_name	no_bikes_difference
1	2014-10-04	Mobile Station @ Bike Fest	96
2	2015-10-02	Zilker Park West	77
3	2015-10-03	Pfluger Bridge @ W 2nd Street	50
4	2015-03-15	Mobile Station	45
5	2018-03-21	Nueces & 26th	44
6	2017-10-07	City Hall / Lavaca & 2nd	44
7	2015-10-10	Pfluger Bridge @ W 2nd Street	44
8	2018-03-01	22nd & Pearl	43
9	2018-04-09	22nd & Pearl	41
10	2018-04-19	22nd & Pearl	41
11	2017-10-14	Pfluger Bridge @ W 2nd Street	41
12	2016-10-02	Pfluger Bridge @ W 2nd Street	41

training_data

```
CREATE OR REPLACE MODEL
```

```
bikeshare_bqml.bike_needed_per_station
```

```
SELECT
```

```
  day,  
  station_name,  
  no_bikes_difference
```

```
FROM
```

```
bikeshare_bqml.training_data;
```

Build and train with **CREATE MODEL**

Row	day	station_name	no_bikes_difference
1	2014-10-04	Mobile Station @ Bike Fest	96
2	2015-10-02	Zilker Park West	77
3	2015-10-03	Pfluger Bridge @ W 2nd Street	50
4	2015-03-15	Mobile Station	45
5	2018-03-21	Nueces & 26th	44
6	2017-10-07	City Hall / Lavaca & 2nd	44
7	2015-10-10	Pfluger Bridge @ W 2nd Street	44
8	2018-03-01	22nd & Pearl	43
9	2018-04-09	22nd & Pearl	41
10	2018-04-19	22nd & Pearl	41
11	2017-10-14	Pfluger Bridge @ W 2nd Street	41
12	2016-10-02	Pfluger Bridge @ W 2nd Street	41

training_data

```
CREATE OR REPLACE MODEL
  bikeshare_bqml.bike_needed_per_station
OPTIONS( MODEL_TYPE='ARIMA_PLUS',
         TIME_SERIES_TIMESTAMP_COL='day',
         TIME_SERIES_DATA_COL='no_bikes_difference',
         TIME_SERIES_ID_COL='station_name' ) AS
SELECT
  day,
  station_name,
  no_bikes_difference
FROM
  bikeshare_bqml.training_data;
```

Build and train with **CREATE MODEL**

By specifying
`TIME_SERIES_ID_COL`,
this query will train
multiple time-series models,
one for every *station_name*.

```
CREATE OR REPLACE MODEL
  bikeshare_bqml.bike_needed_per_station
OPTIONS( MODEL_TYPE='ARIMA_PLUS',
         TIME_SERIES_TIMESTAMP_COL='day',
         TIME_SERIES_DATA_COL='no_bikes_difference',
         TIME_SERIES_ID_COL='station_name' ) AS
SELECT
  day,
  station_name,
  no_bikes_difference
FROM
  bikeshare_bqml.training_data;
```

Build and train with **CREATE MODEL**

For each item:

BigQuery ML creates up to
42 candidate models using
Auto ARIMA **per item_name**,
and then the best model is
automatically chosen for you.

```
CREATE OR REPLACE MODEL
  bikeshare_bqml.bike_needed_per_station
OPTIONS( MODEL_TYPE='ARIMA_PLUS',
         TIME_SERIES_TIMESTAMP_COL='day',
         TIME_SERIES_DATA_COL='no_bikes_difference',
         TIME_SERIES_ID_COL='station_name' ) AS

SELECT
  day,
  station_name,
  no_bikes_difference
FROM
  bikeshare_bqml.training_data;
```

How time series models work in BigQuery ML

How do time series models work in BigQuery ML?

Multiple components used in time series model creation pipeline:

1. **Pre-processing**

- Missing value imputation
- De-duplication of timestamps
- Identifying spike anomalies/abrupt changes

2. **Holiday effects**

- Spike/dip anomalies during holidays will no longer be treated as anomalies

3. [Seasonal and trend decomposition](#)

4. Trend modeling with **ARIMA** and [Auto ARIMA](#)

You can forecast multiple products in the same query

Evaluating the time-series model with BigQuery ML

Evaluating the model

```
SELECT
  *
FROM
  ML.EVALUATE(MODEL bikeshare_bqml.bike_needed_per_station);
```

Row	station_name	non_seasonal_p	non_seasonal_d	non_seasonal_q	has_drift	log_likelihood	AIC	variance	seasonal_periods
1	10th & Red River	2	1	1	false	-352.08314256413024	712.16628512826048	0.65972218314073439	WEEKLY
2	10th/Red River	0	1	3	false	-186.36717943560359	380.73435887120718	0.12345573743436936	WEEKLY
3	11th & Salina	0	1	2	false	-463.37955276449378	932.75910552898756	1.4325768700061072	NO_SEASONALITY
4	11th & San Jacinto	2	1	1	false	-2633.7305927177326	5275.4611854354653	2.6074711977242413	YEARLY
5	11th/Congress @ The Texas Capitol	2	1	1	false	-2336.9663787202926	4681.9327574405852	6.163350348848792	NO_SEASONALITY
6	11th/Salina	2	1	3	true	340.95018494700469	-667.90036989400937	0.024270988609260982	WEEKLY

Explorer

+ ADD DATA

<

Type to search

?

Viewing pinned projects.

bikeshare_bqml

Models (1)

bike_needed_per_station

training_data

MORE RESULTS

bitcoin_blockchain

cardinalhealth

ch03

ch04

ch05

ch05eu

ch07

ch07eu

*Unsaved ...y 4

*Unsaved ...y 5

bike_nee... ion

+

?

bike_needed_per_station

QUERY MODEL

DELETE MODEL

EXPORT I

DETAILS

TRAINING

EVALUATION

SCHEMA

Time Series ID	Non Seasonal P ?	Non Seasonal D ?	Non Seasonal Q ?	Has Drift	Has Spikes And Dips	Has Holiday Effect	Has Step Changes	Log Likelihood	AIC	Variance	Seasonal Period
10th & Red River	2	1	1	False	True	False	False	-352.083	712.166	0.66	Weekly
10th/Red River	0	1	3	False	True	False	True	-186.367	380.734	0.123	Weekly
11th & Salina	0	1	2	False	True	False	True	-463.38	932.759	1.433	No Seasonality
11th & San Jacinto	2	1	1	False	True	False	False	-2,633.731	5,275.461	2.607	Yearly
11th/Congress @ The Texas Capitol	2	1	1	False	True	False	False	-2,336.966	4,681.933	6.163	No Seasonality
11th/Salina	2	1	3	False	True	False	False	340.95	-667.9	0.024	Weekly, Yearly
11th/Salina	0	0	2	False	True	False	False	-52.282	112.563	0.151	Weekly
11th/San Jacinto	1	1	2	False	True	False	False	-1,620.836	3,251.672	1.489	No Seasonality

Predicting forecasted values with **BigQuery ML**

Making forecasts with **ML.FORECAST**

```
SELECT
  *
FROM
  ML.FORECAST(MODEL bikeshare_bqml.bike_needed_per_station,
               STRUCT(30 AS horizon,
                      0.90 AS confidence_level)
               )
```

Making forecasts with ML.FORECAST

JOB INFORMATION		RESULTS	JSON	EXECUTION DETAILS	EXECUTION GRAPH	PREVIEW	
Row	station_name	forecast_timestamp	forecast_value	standard_error	confidence_level	prediction_interval_lower_bound	prediction_interval_upper_bound
1	10th & Red River	2019-05-31 00:00:00 UTC	0.79347714682286519	0.81224509726843519	0.9	-0.54109935922660379	2.1280536528723344
2	10th & Red River	2019-06-01 00:00:00 UTC	0.48246465601698496	1.1287923745057034	0.9	-1.3722215566659859	2.3371508686999558
3	10th & Red River	2019-06-02 00:00:00 UTC	0.36164338514297278	1.2977083360467179	0.9	-1.7705837748815265	2.4938705451674723
4	10th & Red River	2019-06-03 00:00:00 UTC	0.21456295031131778	1.393322874200194	0.9	-2.0747657012279133	2.5038916018505493
5	10th & Red River	2019-06-04 00:00:00 UTC	0.57609575976355409	1.4498811996552541	0.9	-1.8061622460157529	2.9583537655428609
6	10th & Red River	2019-06-05 00:00:00 UTC	0.839295709550217	1.4845010151600828	0.9	-1.599845117155791	3.2784365362562253
7	10th & Red River	2019-06-06 00:00:00 UTC	0.19713001474876041	1.5063237093004658	0.9	-2.2778670516518944	2.6721270811494153
8	10th & Red River	2019-06-07 00:00:00 UTC	-0.16759334475168103	1.5204715054994704	0.9	-2.6658362472713413	2.3306495577679791
9	10th & Red River	2019-06-08 00:00:00 UTC	-0.32075747397031323	1.529912556702596	0.9	-2.8345126955799551	2.1929977476393288
10	10th & Red River	2019-06-09 00:00:00 UTC	-0.27061735292077682	1.53640941229936	0.9	-2.7950473706871404	2.253812664845587
11	10th & Red River	2019-06-10 00:00:00 UTC	-0.28156790803766046	1.541029412068464	0.9	-2.8135889142513975	2.2504530981760764

Visualizing the forecasts in a dashboard with Data Studio

Preparing the data for visualization

Combine
historical data

with

forecasted values
and intervals

Row	day	station_name	history_value	forecast_value	prediction_interval_lower_bound	prediction_interval_upper_bound
1	2015-04-24	Shop	0	null	null	null
2	2015-02-18	Shop	0	null	null	null
3	2020-01-29	3rd/West	0	null	null	null
4	2021-03-18	3rd/West	0	null	null	null
5	2020-06-18	3rd/West	0	null	null	null
6	2019-06-21	3rd/West	0	null	null	null
7	2021-11-03	3rd/West	0	null	null	null
8	2021-09-03	3rd/West	0	null	null	null

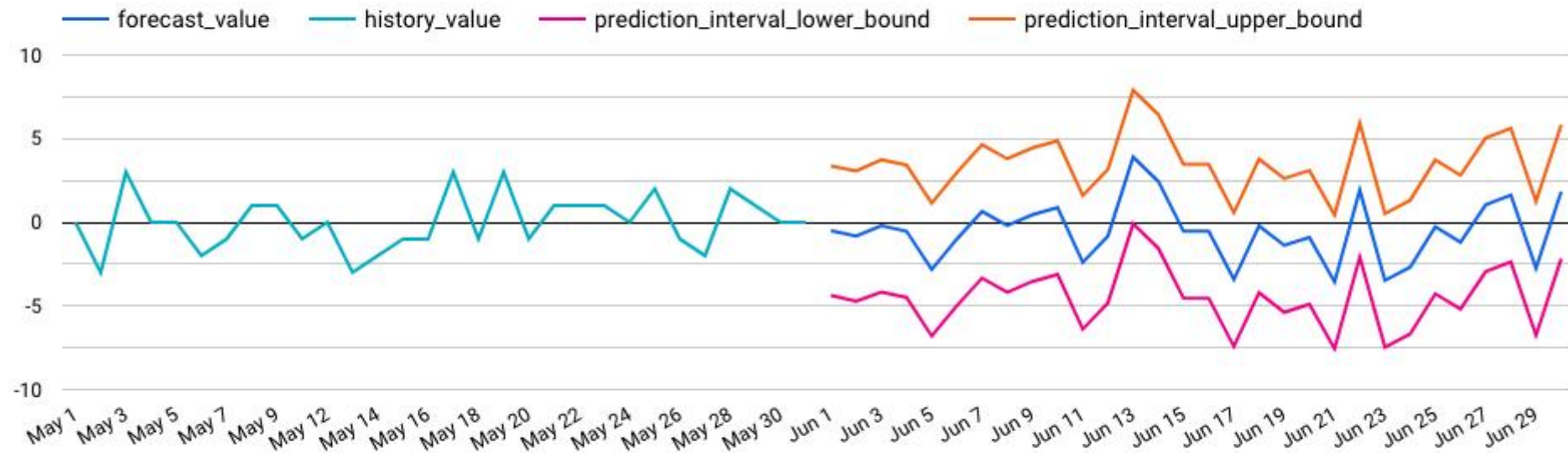
Row	day	station_name	history_value	forecast_value	prediction_interval_lower_bound	prediction_interval_upper_bound
1	2026-08-28	Repair Shop	null	-1.0	-3.4185352834453182	1.4185352834453182
2	2026-05-16	Repair Shop	null	-1.0	-3.3778847364890492	1.3778847364890492
3	2026-02-01	Repair Shop	null	-1.0	-3.3365270644238776	1.3365270644238776
4	2025-10-20	Repair Shop	null	-1.0	-3.2944240291504383	1.2944240291504383
5	2025-07-08	Repair Shop	null	-1.0	-3.2515338168236787	1.2515338168236787
6	2025-03-26	Repair Shop	null	-1.0	-3.2078105514419195	1.2078105514419195
7	2024-12-12	Repair Shop	null	-1.0	-3.1632037199061811	1.1632037199061811
8	2024-08-30	Repair Shop	null	-1.0	-3.1176574880215191	1.1176574880215191

Preparing the data for visualization

Use **UNION ALL** to combine historical and forecasted values into a single query

```
CREATE OR REPLACE VIEW bikeshare_bqml.combine AS (  
  SELECT  
    day, station_name AS station_name,  
    no_bikes_difference AS history_value,  
    NULL AS forecast_value,  
    NULL AS prediction_interval_lower_bound,  
    NULL AS prediction_interval_upper_bound  
  FROM  
    bikeshare_bqml.training_data  
  
  UNION ALL  
  
  SELECT  
    EXTRACT(DATE FROM forecast_timestamp) AS day,  
    station_name AS station_name,  
    NULL AS history_value,  
    forecast_value,  
    prediction_interval_lower_bound,  
    prediction_interval_upper_bound  
  FROM  
    ML.FORECAST(MODEL bikeshare_bqml.bike_needed_per_station,  
      STRUCT(30 AS horizon, 0.9 AS confidence_level))  
  ORDER BY day  
)
```

Bike Share Demand Forecasting



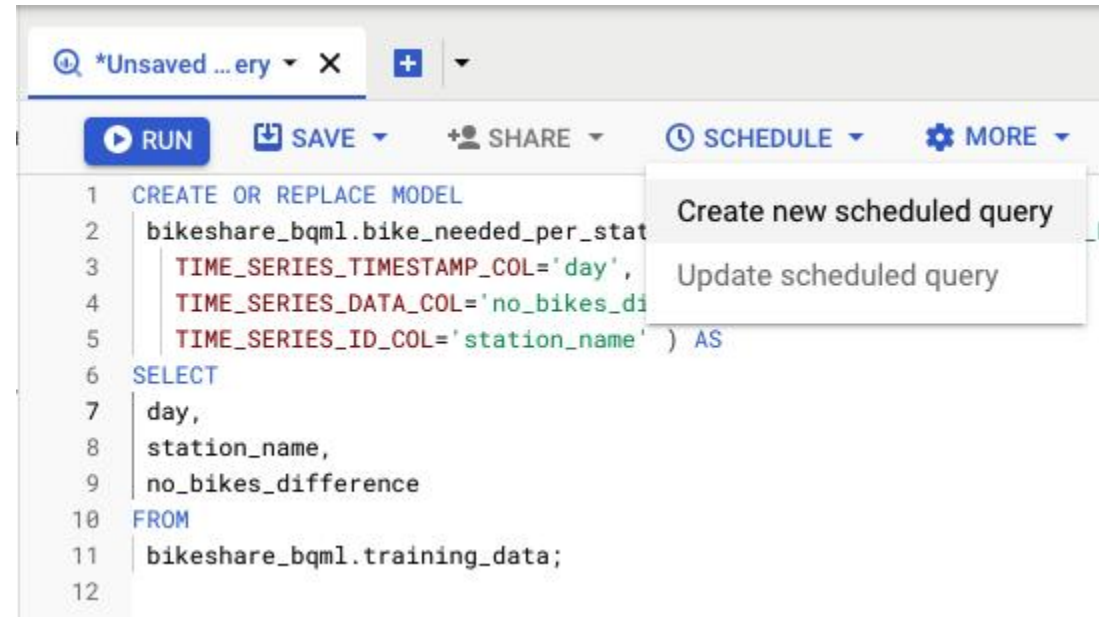
	station_name ^	history_val...
1.	Palmer Auditorium	1,353

1 - 1 / 1 < >

Scheduling and automating model retraining with scheduled queries

Model retraining

1. Create a new scheduled query.



Model retraining

2. Fill in settings for the new scheduled query.

New scheduled query

✔ Settings valid.

Details and schedule

Name for scheduled query *

Re-train demand forecasting model

Schedule options

Repeats *

Weekly

Select days *

Thursday

☒ Start now

☐ Start at set time

Start date and run time

7/7/22, 12:15 PM

BST

📅

☐ End never

☒ Schedule end time

End date *

7/14/22, 12:00 AM

BST

📅

📘

Every Thu at 12:15:22 Europe/London, ending on 00:00:00 Europe/London

Model retraining

3. View your scheduled query

Scheduled queries [+ CREATE SCHEDULED QUERY](#)

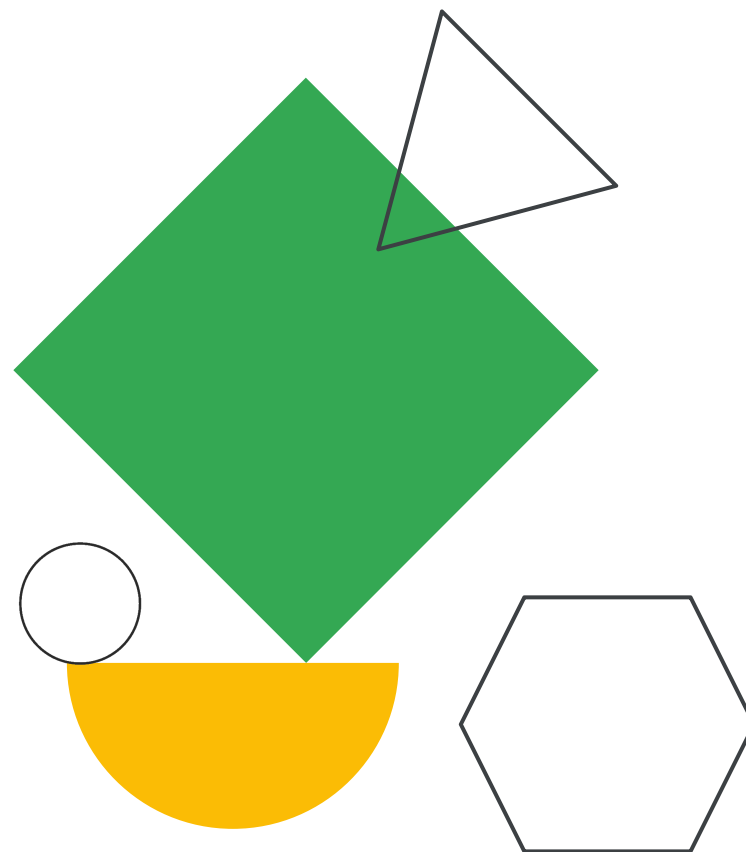
 Filter

 Filter transfer configs

	Display name	Source	Schedule (UTC)	Region	Destination dataset	Next scheduled	UTC+1 ▾	↑	Actions
	Re-train demand forecasting model	Scheduled Query	every thu 11:15	us		None			

Demo (Optional)

Demand Forecasting: Iowa
Liquor Sales





BigQuery ML Explainability

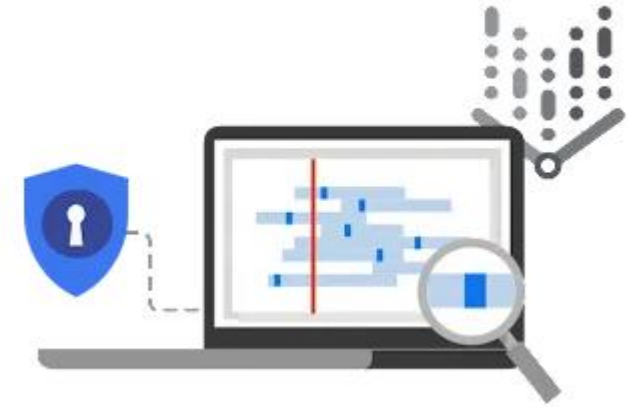
What is Explainable AI ?

- Explainable AI (XAI) is a set of tools and frameworks that are natively integrated with several Google products and services.
- XAI helps you understand and interpret predictions made by your machine learning models.



Why use explainable AI ?

- Verify that the model is behaving as expected.
- Recognize biases in your models.
- Inform ways to improve your model and your training data.
- Help others understand your models' behavior.



Local versus global explainability

Local explainability

Returns feature attribution values for each explained example. These values describe how much a particular feature affected the prediction relative to the baseline prediction.

Global explainability

Returns the feature's overall influence on the model, often obtained by aggregating the feature attributions over the entire dataset. A higher absolute value indicates that the feature had a greater influence on the model's predictions.

Explainable AI offerings in BigQuery ML

- ML.EXPLAIN_PREDICT
- ML.GLOBAL_EXPLAIN
- ML.EXPLAIN_FORECAST
- ML.ADVANCED_WEIGHTS
- ML.FEATURE_IMPORTANCE



Build and train with **CREATE MODEL**

```

1 CREATE OR REPLACE MODEL `ecommerce.classification_model`
2 OPTIONS
3 (
4   model_type='logistic_reg',
5   labels = ['will_buy_on_return_visit']
6 )
7 AS
8 #standardSQL
9 SELECT
10  * EXCEPT(fullVisitorId)
11 FROM
12  # features
13  (SELECT
14    fullVisitorId,
15    IFNULL(totals.bounces, 0) AS bounces,
16    IFNULL(totals.timeOnSite, 0) AS time_on_site
17  FROM
18    `data-to-insights.ecommerce.web_analytics`
19  WHERE
20    totals.newVisits = 1
21    AND date BETWEEN '20160801' AND '20170430') # train on first 9 months
22 JOIN
23  (SELECT
24    fullvisitorid,
25    IF(COUNTIF(totals.transactions > 0 AND totals.newVisits IS NULL) > 0, 1, 0) AS will_buy_on_return_visit
26  FROM
27    `data-to-insights.ecommerce.web_analytics`
28  GROUP BY fullvisitorid)
29 USING (fullVisitorId)
30 ;

```

Review query statement.

```
1 SELECT
2   * EXCEPT(fullVisitorId)
3 FROM
4   # features
5   (SELECT
6     fullVisitorId,
7     IFNULL(totals.bounces, 0) AS bounces,
8     IFNULL(totals.timeOnSite, 0) AS time_on_site
9   FROM
10    `data-to-insights.ecommerce.web_analytics`
11  WHERE
12    totals.newVisits = 1
13    AND date BETWEEN '20160801' AND '20170430') # train on first 9 months
14 JOIN
15   (SELECT
16     fullvisitorid,
17     IF(COUNTIF(totals.transactions > 0 AND totals.newVisits IS NULL) > 0, 1, 0) AS will_buy_on_return_visit
18   FROM
19     `data-to-insights.ecommerce.web_analytics`
20   GROUP BY fullvisitorid)
21 USING (fullVisitorId)
22 ;
```

Insert the query statement
into ML.EXPLAIN PREDICT
function.

```

1 SELECT
2   *
3 FROM
4   ML_EXPLAIN_PREDICT(MODEL `ecommerce.classification_model`,
5   (
6   SELECT
7     * EXCEPT(fullVisitorId)
8   FROM
9     # features
10    (SELECT
11      fullVisitorId,
12      IFNULL(totals.bounces, 0) AS bounces,
13      IFNULL(totals.timeOnSite, 0) AS time_on_site
14    FROM
15      `data-to-insights.ecommerce.web_analytics`
16    WHERE
17      totals.newVisits = 1
18      AND date BETWEEN '20160801' AND '20170430') # train on first 9 months
19    JOIN
20    (SELECT
21      fullvisitorid,
22      IF(COUNTIF(totals.transactions > 0 AND totals.newVisits IS NULL) > 0, 1, 0) AS will_buy_on_return_visit
23    FROM
24      `data-to-insights.ecommerce.web_analytics`
25    GROUP BY fullvisitorid)
26    USING (fullVisitorId)
27  ), STRUCT(3 AS top_k_features))
28

```

Review results

predicted_will_buy_on_return_visit	probability	top_feature_attributions	baseline_prediction_value	prediction_value	approximation_error	bounces	time_on_site	will_buy_on_return_visit
0	0.9930272752	{ "top_feature_attributions": [{ "feature": "bounces", "attribution": "-0.56954676580603514" }], { "feature": "time_on_site", "attribution": "-0.1003191814728472" }] }	-4.288886097	-4.958752044	0	1	0	0
0	0.9930272752	{ "top_feature_attributions": [{ "feature": "bounces", "attribution": "-0.56954676580603514" }], { "feature": "time_on_site", "attribution": "-0.1003191814728472" }] }	-4.288886097	-4.958752044	0	1	0	0

Lab (45:00)

Predict Bike Trip Duration with a Regression Model in BigQuery ML

45:00

Questions?

